FlintNDE: High-precision local-series transport for matrix differential equations

FlintNDE project

Draft of August 14, 2026

Abstract

We present `FlintNDE`, a Python package built on FLINT for arbitrary-precision numerical continuation of first-order matrix differential equations. Its first core function transports a boundary vector along a user path by local series, with automatic ordinary/regular-singular dispatch, exact-first Frobenius and logarithm analysis, exact Lee–Moser projector balances, certified exponential sectors, and an independent refinement chain. Its second core function repeats that solver at fixed regulator values and reconstructs a semi-analytic Laurent series whose powers are analytic and whose vector coefficients are high-precision numerical values. This strategy avoids carrying a symbolic regulator expansion through every relay point. Exact Gaussian-rational matrices provide an automatic finite and infinite singularity inventory; general analytic evaluators use Cauchy–DFT coefficient extraction without symbolic differentiation. A higher-order pole initially certifies only that the supplied basis is non-Fuchsian. FlintNDE then applies exact projector balances built from the leading two Laurent matrices, a strictly decoupled exponential-times-power–log ansatz, and, for a rank-one irregular point with simple distinct leading roots, a start-only formal exponential series with fixed-order five-term diagnostics. Ramified or algebraic formal reductions, Stokes matrices, and arbitrary Levelt–Turrittin problems remain fail closed.

1 Introduction

1.1 Flat-space Feynman integrals

Perturbative quantum-field-theory amplitudes are expressed through families of loop integrals. Integration-by-parts (IBP) identities reduce a family to finitely many master integrals [1]; differentiating those masters with respect to a kinematic, mass, or auxiliary parameter and reducing again gives a closed first-order matrix differential equation [2]. The remaining numerical problem is to impose boundary data, cross singular regions, and evaluate the solution at physical points.

Several public approaches expose complementary versions of this IBP–DE pipeline. `DiffExp` joins one-dimensional local series along line segments in phase space [3]. AMFlow combines IBP reduction, auxiliary-mass differential equations, systematic large-mass boundary data, and high-precision numerical transport to compute general dimensionally regulated loop integrals [4, 5]. FlintNDE adopts one specific AMFlow component: it solves the complete problem at several fixed regulator values and numerically reconstructs the Laurent coefficients. In a long path with many relay points, this avoids propagating an analytic regulator jet through every intermediate matrix multiplication, which otherwise causes a rapid increase in algebraic complexity and memory use. The reconstruction remains numerical: it is checked by independent regulator samples, not presented as an exact symbolic proof.

AmpRed provides another general IBP–DE route in the Feynman-parameter space [6]. Its parameter integrals are organized in the Lee–Pomeransky (LP) representation [7]; parametric IBP relations construct the master system, while asymptotic boundary data are organized with

expansion by regions [8, 9]. FlintNDE does not replace any reduction or region-analysis frontend. It consumes the resulting one-variable matrix system and certified boundary data.

1.2 de Sitter and cosmological integrals

Massive fields during inflation leave characteristic non-Gaussian signals, motivating the quasi-single-field and cosmological-collider programs [10, 11, 12]. Here we mention only techniques that directly construct IBP relations and differential equations. The first two papers of the dSIBP series formulate generalized IBP relations for time integrals and solve the resulting systems, including multivariate hypergeometric and $\mathrm{d}\log$ forms [13, 14]. The loop extension treats both the time integrations and the loop-momentum integration inside one parity-split IBP/DE system [15].

A complementary energy-space route converts time integrations to energy variables and derives differential equations through kinematic flow [16, 17]. Cohomological IBP in the same type of energy representation and the massive-correlator extension provide related realizations [18, 19]. These applications motivate a solver that is independent of whether the matrix originated from a flat-space loop family, a cosmological time integral, or a combined time–loop-momentum family.

1.3 Shared numerical problem and package scope

Although their integral representations differ, the flat-space and cosmological systems above lead to the same downstream numerical task. One must continue a vector solution of

$$\frac{\mathrm{d}Y(s)}{\mathrm{d}s} = A(s,\epsilon)Y(s) \tag{1}$$

from certified boundary data to a target point, while preserving the local solution space at singular points and controlling truncation and working-precision errors along a path. Long continuation chains make this distinction important: ordinary Taylor transport, Frobenius power–log transport, and regulator reconstruction solve different problems and must not be interchanged.

FlintNDE begins after the reduction stage. Its contract is a one-variable first-order matrix DE, boundary data, and a direct point list. It neither generates IBP identities nor assumes a particular master-integral notation. For exact Gaussian-rational input it determines finite and infinite singularities and performs exact-first indicial, Jordan, and resonance tests; for general analytic input it evaluates ordinary-point Taylor coefficients numerically by Cauchy–DFT sampling. Both routes use FLINT arbitrary-precision complex ball arithmetic for the actual transport and an independent refinement chain for error control.

The package exposes two workflows. The first transports a solution vector through ordinary and regular singular points, using a unified power–log recurrence whenever the local structure requires logarithms. The second calls the same transport repeatedly at fixed regulator values and reconstructs the numerical coefficients of a Laurent series. A pole of order greater than one is first reported as a non-Fuchsian input basis. The local dispatcher then applies exact Lee–Moser projector balances and, if a higher pole remains, either mutually decoupled scalar exponential sectors or the restricted rank-one formal recurrence described in Sec. 6.4. The latter is an initial-boundary evaluator, not a Stokes connection solver. Ramified, algebraic, and arbitrary Levelt–Turrittin/Stokes problems remain outside the present implementation. The next two sections give the complete workflows; installation, examples, and the mathematical and software details follow afterward.

2 Core function I: numerical matrix-DE transport

The basic function continues an N-component boundary vector through a direct point list using FLINT arbitrary-precision complex ball arithmetic. It accepts exact $\mathbb{Q}(i)(s)$ matrices or gen-

eral analytic matrix evaluators. Exact structure is used where it changes the solution space—singularities, indicial roots, Jordan blocks, and resonance—while ordinary-point coefficients and transport are numerical.

Figure 1 gives the complete basic workflow. Logarithm analysis appears as one node here and is expanded separately in Fig. 3.

3 Core function II: semi-analytic regulator-series reconstruction

The second function reconstructs

$$F(\epsilon) = \sum_{n=n_{\min}}^{m} f_n \epsilon^n + O(\epsilon^{m+1}), \tag{2}$$

where the integer powers are analytic information and each f_n is an arbitrary-precision numerical vector. Both m and an upstream symbolically certified $n_{\min}$ are required; the sample grid, working precision, and surplus fitted orders are automatic unless overridden. Every regulator sample calls the complete basic solver of Fig. 1; no independent solution-fitting DE engine exists.

4 Installation and dependencies

The runtime requirements are only Python ≥ 3.10 and `python-flint>=0.6`. After publication, install from the package index with

```
python -m pip install FlintNDE
```

or install a downloaded source tree once with

```
python -m pip install path/to/package-FlintNDE/versions/FlintNDE-0.4.0
```

Editable development mode uses the same command with `-e`. A calling script may then live in any directory and simply `import flintnde`; the package source is not copied beside it. When the caller passes `initialize_output_layout(__file__)`, all generated files are rooted beside that calling script, never in the installed package. Importing the package initializes the global Acb context to 200 decimal working digits plus 32 guard bits. Calling `configure_working_precision()` restores that default, while an explicit positive integer overrides it.

The source tree also provides a standard Wolfram Language package. Add the 0.4.0 version root to `$Path`, then evaluate

```
Needs["FlintNDE`"];
system = FlintNDERationalSystem[{{1/(x-1)}}, x];
plan = FlintNDEPlanPath[system, 0, {1/2},
  "WorkingPrecisionDigits" -> 100];
result = FlintNDEExecutePath[system, {1}, plan,
  "WorkingPrecisionDigits" -> 100];
```

The planning default is `"SingularityMode" -> "Avoid"`; explicit `"SingularityJump"` mode carries the branch warning. These are the only accepted mode values. Messages use English by default and `MessageLanguage -> "CN"` selects Chinese.

Independent fixed-regulator jobs use `FlintNDEEvaluateEpBatch`, with the option `ParallelTaskCount -> 12`. The default upper bound is 12 worker processes; the effective count is the smaller of that bound and the number of jobs, and queued jobs start automatically as workers finish. The Python counterparts are `run_ep_tasks(..., parallel_task_count=12)` and the same `parallel_task_count` option of `reconstruct_series_solution`. This outer process count is independent of python-flint's in-process `ctx.threads` setting.

The complete input syntax and every optional parameter are listed in Sec. 6.6.

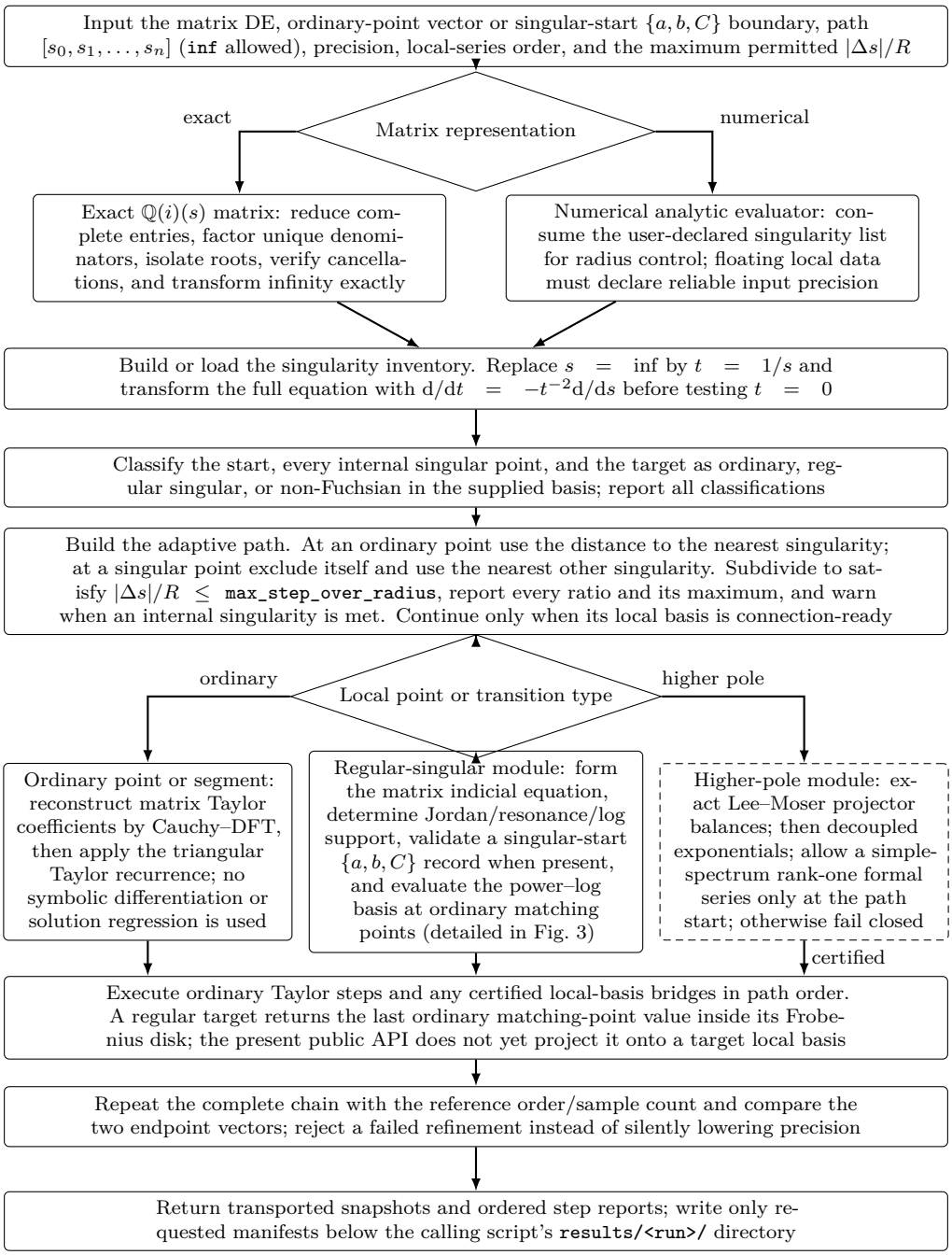

Figure 1: Basic numerical matrix-DE workflow. Input formats and output layout are detailed in Sec. 6.6; exact/numerical representations and ordinary recurrences in Sec. 6.1; singularity inventory and infinity in Sec. 6.1.2; path construction and each $|\Delta s|/R$ report in Sec. 6.1.3; matrix indicial data in Sec. 6.2; the log node in Sec. 6.3; the high-pole module and its fail-closed boundary in Sec. 6.4; and transport/refinement in Sec. 6.6.3.

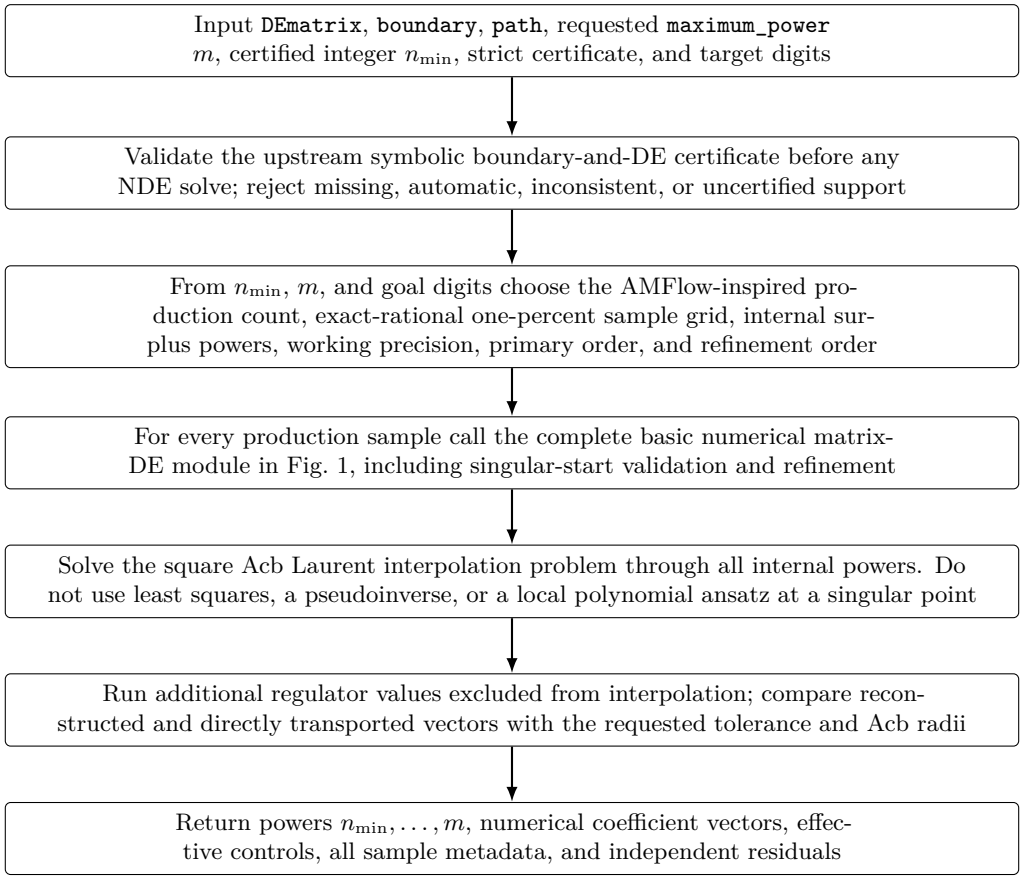

Figure 2: Semi-analytic regulator-series workflow. The basic NDE node is exactly Fig. 1 and is not expanded again. The AMFlow error estimate, automatic ϵ scale, surplus orders, precision rules, support certificate, square interpolation, and validation are detailed in Sec. 6.5; the complete optional parameter list is in Table 6 of Sec. 6.6.

5 Examples

5.1 Certified Laurent reconstruction

The executable `examples/ep_series_reconstruction.py` uses the zero matrix DE and the analytically known boundary

$$Y(0,\epsilon) = \epsilon^{-1} + 2 + 3\epsilon + 4\epsilon^2 + 5\epsilon^3 + 6\epsilon^4. \tag{3}$$

The upstream example certificate therefore states $n_{\min} = -1$. FlintNDE validates that certificate before any solve, but does not infer it from sampled values. The user requests only the finite part ($m = 0$); the first square fit includes two additional powers. With the deliberately strict independent-point tolerance, a failed first round appends two production points while reusing every old production and validation value. The accepted result recovers the pole coefficient 1 and finite part 2. The example uses `parallel_task_count=12`; the process pool automatically reduces the effective count when fewer regulator tasks are available and queues excess tasks otherwise.

5.2 de Sitter backend usage

MadStree provides the physical de Sitter example at the upstream layer: it derives the symbolic boundary and dlog DE, certifies the regulator support, and then calls this package. FlintNDE does not duplicate dS topology, master ordering, or normalization in a backend-only example.

5.3 Schwarzschild two-ended radial system

The retained cross-domain example starts from the odd-parity Regge–Wheeler function u_o^- in units $2M = 1$:

$$\frac{\mathrm{d}^2 u_o^-}{\mathrm{d}x^2} + \frac{1}{x(x-1)}\frac{\mathrm{d}u_o^-}{\mathrm{d}x} + \left[\frac{\omega^2 x^2}{(x-1)^2} - \frac{\ell(\ell+1)x - 3}{x^2(x-1)}\right] u_o^- = 0. \tag{4}$$

The even-parity function u_o^+ need not be assigned a second explicit equation here: at fixed (ℓ, ω) it is mapped to u_o^- and its first derivative by the standard Chandrasekhar–Darboux transformation [20]. The prefactor

$$u_o^- = x^{\alpha_0}(x-1)^{\alpha_1 i\omega} u, \qquad (\alpha_0, \alpha_1) = (3, -1), \tag{5}$$

removes the second-order finite poles. Writing $U = (u, u')^T$ and diagonalizing the constant part with

$$S_\infty = \begin{pmatrix} 1 & 1 \\ i\omega & -i\omega \end{pmatrix}, \qquad U = S_\infty a, \tag{6}$$

gives the explicitly organized first-order system

$$\frac{\mathrm{d}a}{\mathrm{d}x} = \left[\Lambda_\infty + \frac{R_0}{x} + \frac{R_H}{x-1}\right] a, \qquad \Lambda_\infty = \begin{pmatrix} i\omega & 0 \\ 0 & -i\omega \end{pmatrix}, \tag{7}$$

where, with $\lambda = (\ell-1)(\ell+2)/2$ and $d = (\lambda - 2)/\omega$,

$$R_0 = \begin{pmatrix} -5 + id & id \\ 5 - id & -id \end{pmatrix}, \tag{8}$$

$$R_H = \begin{pmatrix} 2 + 2i\omega - id & 3 - id \\ -2 - 2i\omega + id & -3 + id \end{pmatrix}. \tag{9}$$

Thus no unreduced companion-matrix expression is needed by the example.

The physical meaning of the zero leading series coefficient must be read after the known endpoint factors are removed. Since $r_* \sim \log(x-1)$ at the horizon, the ingoing plane wave

$e^{-i\omega r_*}$ is absorbed by Eq. (5); its remaining Frobenius series has indicial leading power zero. At infinity, after extracting $e^{\pm i\omega r_*}$ and the corresponding algebraic power in $z = 1/x$, the outgoing or incoming analytic remainder likewise starts at power zero. These zero roots label the regular remainders of asymptotic plane waves, not constant physical waves.

For the tabulated quasinormal frequency, the script starts literally at $x = 1$ once and at `inf` once, transports the selected branches to a common matching point, and decomposes each vector in the opposite endpoint basis [21, 22]. The forbidden-to-allowed ratios are

Frequency	horizon → infinity	infinity → horizon
tabulated quasinormal frequency	2.4549×10^{-15}	3.7235×10^{-15}
relative shift 10^{-3}	3.9002×10^{-3}	5.9175×10^{-3}

Infinity is first mapped to $z = 1/x = 0$. Its A_{-2} eigenvalues are simple and distinct, so Eq. (36) constructs the outgoing and incoming branches and the nearest-root-gap rule with $q = 3$ gives the first ordinary point $x \simeq 74.20$. At reference order 55 the outgoing/incoming five-order block ratios are approximately 0.02037 and 0.02210; neither branch has reached its least term in the available range. Their local vectors agree with the independently implemented scalar second-order recurrence to 1.2×10^{-42} and 5.1×10^{-42}, respectively; the separate comparison at $x = 28$ remains better than 3×10^{-18}. The more distant start amplifies Cauchy–DFT aliasing between stiff exponential branches, so this example uses sampling-radius fraction 0.40; the maximum primary/reference transport difference is then 3.75×10^{-22}. Hence the first table row is consistent with zero at the demonstrated refinement scale, while the nonzero shifted row is a counterfactual boundary mismatch. This is a formal/Gevrey infinity expansion, not a convergent Frobenius disk. The example tests a radial connection problem only and introduces no integral-family assumption.

6 Technical details

6.1 Matrix differential equations and ordinary points

We use the column-vector convention

$$\frac{\mathrm{d}}{\mathrm{d}z}Y(z) = A(z)Y(z), \qquad A(z) \in \mathbb{C}^{N\times N}. \tag{10}$$

At an ordinary point c, write $s = z - c$ and

$$A(c+s) = \sum_{m\geq 0} A_m(c)s^m, \qquad Y(c+s) = \sum_{n\geq 0} Y_n(c)s^n. \tag{11}$$

Substitution in Eq. (10) gives the triangular recurrence

$$Y_{n+1}(c) = \frac{1}{n+1}\sum_{m=0}^{n} A_m(c)Y_{n-m}(c). \tag{12}$$

The recurrence advances either one boundary vector or a complete fundamental matrix. The former is cheaper when only one physical boundary condition is required.

6.1.1 General analytic matrix coefficients

The default backend does not assume that $A(z)$ has only simple poles. If A is analytic on and inside the circle $|s| = r$, Cauchy's formula gives

$$A_m(c) = \frac{1}{2\pi r^m}\int_0^{2\pi} A(c + re^{\mathrm{i}\theta})e^{-\mathrm{i}m\theta}\,\mathrm{d}\theta. \tag{13}$$

With K equally spaced samples this becomes the discrete Fourier projection

$$A_m^{(K)}(c) = \frac{1}{Kr^m} \sum_{j=0}^{K-1} A(c + r\xi_K^j)\xi_K^{-jm}, \qquad \xi_K = e^{2\pi \mathrm{i}/K}. \tag{14}$$

This is a coefficient reconstruction from numerical matrix evaluations, not symbolic differentiation and not a regression fit for the solution. It applies equally to rational, algebraic, and user-supplied analytic matrix evaluators, provided the sampling circle lies in their common analytic domain.

For rational matrices, direct series division can later be supplied as an optional coefficient generator. Equation (14) remains the general default and a common validation route. The truncation and Fourier-aliasing error is monitored by a second transport with higher Taylor order and more Cauchy samples.

6.1.2 Exact singularity inventory and infinity

For an exact Gaussian-rational input, each complete matrix entry is first reduced in $\mathbb{Q}(i)[s]$ as

$$A_{ij}(s) = \frac{N_{ij}(s)}{D_{ij}(s)}, \qquad \gcd(N_{ij}, D_{ij}) = 1. \tag{15}$$

The finite candidate set is the union of roots of the reduced denominators. A scalar $a + ib$ is stored as two FLINT `fmpq` values, and an N-dimensional complex matrix as two N-dimensional `fmpq_mat` objects; no $2N$ real-block embedding is used. Exact $\mathbb{Q}(i)[s]$ arithmetic performs the polynomial gcd and square-free decomposition. If a square-free factor contains both the exact root zero and algebraic roots outside $\mathbb{Q}(i)$, the factor s is divided out first. Hence the boundary point $s = 0$ retains its exact Gaussian-rational identity and remains eligible for exact Frobenius dispatch, while the remaining roots are isolated as Acb balls. The pole order recorded at a root is the largest order among all entries of the *complete* matrix. A first-order pole of the full matrix is Fuchsian. An order greater than one certifies only `non_fuchsian_input_basis`; it is not, by itself, a basis-invariant proof of an irregular singularity. In particular, inspecting only diagonal entries is invalid, but a higher-order off-diagonal entry may still be lowered by a meromorphic gauge or shearing transformation. Combining and reducing each complete entry before collecting denominators is essential: terms with the same denominator may cancel, and a cancelled candidate must not enter the inventory.

The point at infinity is tested in its own local variable. With

$$t = \frac{1}{s}, \qquad \frac{\mathrm{d}s}{\mathrm{d}t} = -\frac{1}{t^2}, \tag{16}$$

Eq. (10) becomes

$$\frac{\mathrm{d}Y}{\mathrm{d}t} = B(t)Y, \qquad B(t) = -\frac{1}{t^2} A\left(\frac{1}{t}\right). \tag{17}$$

Thus the pole order of the full B at $t = 0$, including the derivative Jacobian, tests whether infinity is ordinary, Fuchsian, or non-Fuchsian in the transformed input basis. For a reduced scalar entry with numerator and denominator degrees n and d, its contribution has pole order $\max(0, n - d + 2)$. Exact divisibility is both faster and stronger than probing $A(p + \delta)$ for small numerical δ; displaced evaluations are useful diagnostics but cannot certify a cancellation or distinguish a small residue from zero.

After building this inventory, the package attempts an exact structural decomposition

$$A(s) = P(s) + \sum_j \frac{R_j}{s - p_j}, \tag{18}$$

where the matrix polynomial P may have any finite degree. The pole locations and residues are derived internally; the caller need not supply a dlog alphabet. The partial-fraction recurrence is selected only if reconstruction of the complete matrix is exact and every finite pole is simple. A higher-order pole or any failed identity check remains on the general rational-matrix route. A pure polynomial system is the valid zero-finite-pole subcase. Thus Eq. (18) is a certified optimization of the generic interface, not a restriction to dlog differential equations.

6.1.3 Path subdivision and precision

At an ordinary point c, let $R(c)$ be the distance to the nearest finite singularity. At a singular point p, let

$$R_{\mathrm{sing}}(p) = \min_{q \neq p} |p - q|, \tag{19}$$

where the point itself is excluded. The package uses

$$|\Delta z| < r < R(c) \tag{20}$$

for every segment. A long path is subdivided according to a fixed fraction of $R(c)$. The primary and reference chains are evaluated independently. Their final relative difference is a truncation diagnostic; it is not confused with the radius carried by an individual FLINT ball. Multi-segment calculations restart from high-precision midpoints to limit interval wrapping and are consequently reported as arbitrary-precision point transport, not as a rigorous enclosure of the complete path.

The convergence radius and the step length are distinct quantities. The singularity set fixes $R(c)$ or $R_{\mathrm{sing}}(p)$; the user option `max_step_over_radius` only imposes $|\Delta z|/R \leq f_{\mathrm{step}}$. Planning and execution are separate operations: a planner consumes raw user points and serializes all ordinary nodes, sample assignments, singularity classifications, and bridge geometry; the executor restores that plan and must not call the planner again.

The default mode is `singularity_mode="avoid"`. If a straight segment contains an internal singularity, planning stops with a structured report naming both the singularity and the affected user-point pair. The caller can instead provide explicit detour points. Any multi-point transport through intermediate nodes is called a jump in this manual. Only the explicit `"singularity_jump"` mode builds a local-basis bridge across a singularity; this operation is called a singularity jump. No Boolean switch or alternate mode spelling is accepted. A singularity jump chooses a branch of a multivalued local solution equivalent to one detour homotopy class; the package reports this responsibility and the caller must confirm the branch. The bridge may use the original Frobenius system, a Moser-reduced Fuchsian system, or a certified exponential-times-power–log sector. A high-pole checkpoint remains fail closed when none of these exact gates supplies a local basis.

When a singularity jump is enabled, entering the step range of a pole creates incoming and outgoing matching nodes. Subsequent user points are inspected in order; the last point still within one admissible pole-centred step becomes the next node. If the first point is already outside that range, one admissible step is taken towards it before ordinary planning resumes. The incoming step is controlled by $R_{\mathrm{sing}}(p)$ of the target singularity, not by the radius of the preceding ordinary point.

The global FLINT precision is

$$p_{\mathrm{bit}} = \lceil p_{\mathrm{dec}} \log_2 10 \rceil + 32. \tag{21}$$

For example, 70 and 100 decimal digits request 265 and 365 bits, and higher decimal requests continue to increase the bit count by the same formula. Segment projection, match ratios, rotation factors, and winding/monodromy geometry are constructed at the current Arb precision rather than in binary64. Serialized plans record `planning_precision_digits`; Acb coordinates

are stored as Arb midpoint–radius– exponent records. Execution at a higher working precision than planning is rejected and requires replanning, because serialization cannot recover discarded information. Increasing only the number of output digits does not change the working precision. The Python import and all three Wolfram entry points default to 200 decimal digits, which gives 697 bits including the guard bits.

Table 1: Execution contract for ordinary and singular path locations.

Path role	Implemented action, radius rule, and returned object
Ordinary start	The input is a finite Acb column. Taylor transport starts at the stated point, and its convergence radius is the distance to the nearest singularity.
Singular start	A regular singularity accepts an exact $\{a, b, C\}$ power–log record; an exponential singularity accepts only an explicit $\{\phi, a, b, C\}$ record. Neither is a finite value at the singularity. The package validates the indicial, exponential, and log branch, evaluates it at the first ordinary matching point, and then starts Taylor transport. The matching distance is limited by R_{sing}, which excludes the singularity itself. For an exponential solution, FlintNDE derives the exact sector from the DE and verifies ϕ; it never infers a missing exponential signature from C.
Ordinary target	The last snapshot is the transported Acb column at the stated target.
Regular-singular target	The executable list ends at an ordinary matching point inside the target Frobenius disk; it never invents a finite value at the singularity. The target classification, singular point, matching point, and match-distance ratio are reported. If the target coordinate carries the save tag and a direct regular-singular power–log basis is available, FlintNDE solves for and writes reusable $\{a, b, C\}$ coefficients. These are local boundary data, not a finite value at the singularity.
Internal regular singularity	The path stores ordinary matching points on both sides and a Frobenius bridge between them. Both match-to-singularity distances are divided by the singularity radius R_{sing}, not by either neighboring ordinary-point radius. The principal Acb log/power branch and both ratios are reported.
Higher-pole checkpoint	The inventory first records `non_fuchsian_input_basis`. The local dispatcher tries exact Lee–Moser projector balances and then the certified decoupled exponential-sector gate. A successful route uses the same two-sided local bridge and returns the original integral basis. Otherwise continuation fails closed; an internal point may instead be avoided with explicit ordinary detour points. A continuation-ready exponential checkpoint tagged by `(coordinate,"save")` writes reusable $\{\phi, a, b, C\}$ data at the bridge.
Infinity	The complete equation is first transformed to $t = 1/s$, including the $-t^{-2}$ Jacobian. The resulting point $t = 0$ then follows the same ordinary, regular-singular, or non-Fuchsian-input-basis rules; its local radius is measured to the nearest other singularity in the transformed variable.

6.1.4 Capability preflight and the fail-closed boundary

Before transport, `build_adaptive_path_plan` applies the same local dispatcher to both endpoints and every internal singular checkpoint. Its `continuation_ready` flag is a capability gate, not an accuracy estimate. If it is false, the plan records the singularity classification, attempted certified route, and the missing structure (for example a ramified or defective formal block, an algebraic extension, or a Stokes connection). The caller must then choose ordinary detour points or supply external local connection data. The numerical transport must not be started on that plan.

This gate applies to every nonordinary checkpoint, not only to a high-order pole. A regular singularity whose center is outside $\mathbb{Q}(i)$, or whose exact indicial, Jordan, or resonance structure is

unsupported by the present power–log constructor, also sets `continuation_ready=False`. The direct path builder reports the point kind and the failed local-basis reason instead of postponing the failure until transport.

A direct call to the path builder enforces the same rule. It raises `LocalReductionError` before transport. In particular, failure of the exact, Lee–Moser, exponential, or rank-one formal gates never falls back to an ordinary Taylor/Cauchy expansion. The sole start-only formal route initializes a solution from the singular boundary to a first ordinary matching point in one Stokes sector; it is not continuation-ready when the same point is internal or the target.

6.1.5 Coordinate-only save tags

A saved path point has no user-supplied name. Its input is exactly `(coordinate, "save")`; for example,

```
path = [(z0, "save"), z1, (z2, "save")]
transport_path_refined(system, boundary, path)
```

The same pair may be used for adaptive-path start, target, or detour coordinates. A singular middle coordinate is accepted only with this tag; it denotes a saved singular checkpoint, not an ordinary detour. Internal `NamedPoint` labels remain routing diagnostics and are not part of the save contract.

On reaching a tagged coordinate, FlintNDE immediately writes `flintnde_save_001.json`, `flintnde_save_002.json`, and so on in path order. The default directory is the caller's `Path.cwd()`; `save_output_directory` may select another caller-owned directory. Only after all transport steps succeed does the package write `flintnde_save_points.json`. Thus a later failure preserves completed point files without claiming a complete summary. A refined transport writes the reference chain only.

At an ordinary point the record contains the coordinate and the transported Acb column. At a direct regular singular start or target it contains reusable $\{a, b, C\}$ terms and `resultType=frobenius_boundary`. A certified decoupled exponential start, target, or middle bridge instead contains $\{\phi, a, b, C\}$ and `resultType=exponential_boundary`. These are boundary data, not $Y(z_*)$. A start-only formal exponential branch may be saved at the start and records `continuationReady=false`; the same method remains forbidden internally or at the target. Saving a Lee–Moser transformed original-basis singularity still requires a certified inverse local jet, and unsupported Stokes connections remain fail closed.

6.2 Regular singular points and the matrix indicial equation

Near $z = 0$, consider a Fuchsian system

$$\frac{\mathrm{d}Y}{\mathrm{d}z} = \left(\frac{R}{z} + \sum_{m \geq 0} A_m z^m \right) Y. \tag{22}$$

For $Y = z^\rho \sum_{n \geq 0} v_n z^n$, the leading equation is

$$(\rho \mathbf{1} - R) v_0 = 0, \qquad \det(\rho \mathbf{1} - R) = 0. \tag{23}$$

Equation (23) is the matrix indicial equation. It is unnecessary to eliminate the system to a scalar higher-order equation before determining its local exponents. For a semisimple nonresonant branch, the higher coefficients obey

$$\big((\rho + n)\mathbf{1} - R\big) v_n = \sum_{m=0}^{n-1} A_m v_{n-1-m}. \tag{24}$$

See Ref. [23] for a Feynman-integral implementation of generalized series and their continuation.

6.3 When logarithms are required

The presence of a repeated indicial root is not, by itself, a criterion for logarithms. There are two distinct mechanisms.

6.3.1 Non-semisimple residue

Suppose a single-root block is $R = \rho\mathbf{1} + N$, with N nilpotent. Already for the leading system one has

$$z^R = z^\rho e^{N \log z} = z^\rho \sum_{q=0}^{\nu-1} \frac{N^q}{q!} (\log z)^q, \tag{25}$$

where $N^\nu = 0$. A Jordan block of length ν therefore produces logarithms up to degree $\nu - 1$. By contrast, a repeated but semisimple eigenvalue has $N = 0$ and produces no log through this mechanism.

6.3.2 Integer-difference resonance

Even for a diagonalizable residue, Eq. (24) becomes singular if $\rho + n$ is another indicial root. Let $P_{\rho+n}$ be the exact spectral projector onto that eigenspace and let b_n denote the right-hand side. A pure-power recurrence exists only if

$$P_{\rho+n} b_n = 0. \tag{26}$$

If this compatibility condition fails, the defect is transferred to a logarithmic coefficient. The package evaluates Eq. (26) with exact rational matrices. The default route never silently substitutes a floating-point threshold for this exact decision; the explicit numerical route described below records its full threshold provenance.

6.3.3 Unified power–log recurrence

Write

$$Y = z^\rho \sum_{n \geq 0} \sum_{q=0}^{q_{\max}} v_{n,q} z^n (\log z)^q. \tag{27}$$

At fixed (n, q) the recurrence has the form

$$\big((\rho + n)\mathbf{1} - R\big) v_{n,q} + (q+1) v_{n,q+1} = \sum_{m=0}^{n-1} A_m v_{n-1-m,q}. \tag{28}$$

The default implementation uses FLINT rational matrices to create an exact manifest containing roots, Jordan nilpotency, spectral projectors, initial eigenvectors, and every active resonance gate. FLINT then evaluates Eq. (28) at the requested precision. Exact classification is the default and never falls back to a threshold decision after an exact failure. Only floating-point input selects the numerical gate, which uses

$$\tau_{\mathrm{rel}} = N\, 10^{-p}, \qquad \tau_{\mathrm{abs}} = \tau_{\mathrm{rel}} \max\left(\max_{i,j} |R_{ij}|, \max_{m,i,j} |(A_m)_{ij}| \right), \tag{29}$$

where N is the system dimension (a conservative rank upper bound) and p is the reliable decimal input precision, not merely the arithmetic working precision. Python binary64 input is capped at 15 decimal digits. Higher-precision decimal or Acb input must carry or declare its reliable precision; otherwise classification fails closed. The user may replace τ_{rel}. Its manifest and mandatory warning report the matrix scale, both thresholds, and the largest zeroed and smallest retained quantities in absolute units and relative to the matrix scale. A zero matrix scale

is reported explicitly, with undefined relative ratios rather than a fabricated number. This threshold route is explicitly numerical rather than an exact certificate. Ambiguous mixed-root defective systems fail closed.

Figure 3 summarizes the implemented decision route. The non-semisimple and resonance mechanisms are cumulative: a Jordan logarithm does not skip the later resonance checks.

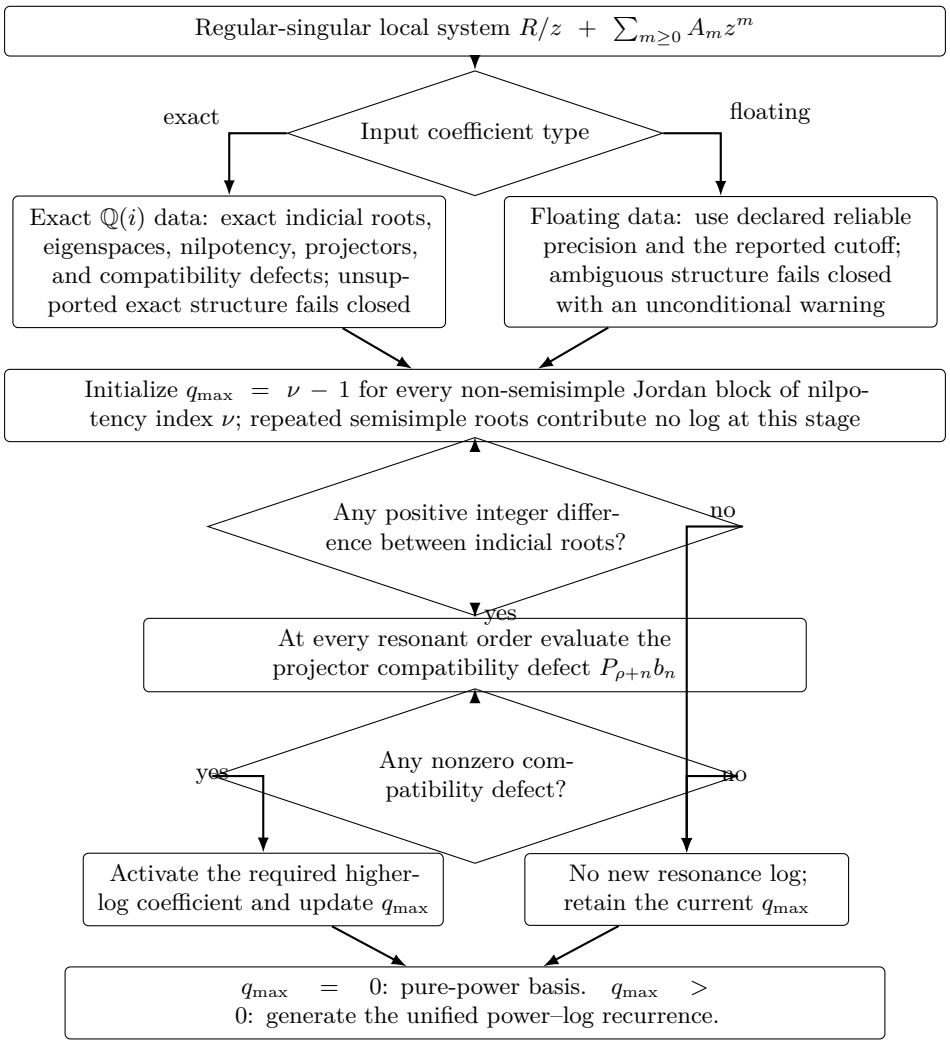

Figure 3: Exact-first and precision-aware numerical routes for deciding logarithmic support.

Once either gate activates a logarithmic sector, the local solution is always generated by Eq. (28). The package has no singular-point path that regresses a pure polynomial solution and thereby drops a logarithmic branch. The Laurent fit in Section 6.5 concerns regulator dependence only and is not a local-solution ansatz.

6.4 Non-Fuchsian input bases and genuine irregularity

The test must use the complete matrix, not only its diagonal. Nevertheless, finding a pole of order two or higher in any entry proves only that the supplied basis is non-Fuchsian. Meromorphic gauge transformations, including shearing transformations, can lower the pole order and can remove a high-order off-diagonal term. FlintNDE therefore applies the following three-stage local dispatcher; the inventory label itself remains conservative.

First it applies the exact Lee–Moser reduction. At a pole of order $p + 1 > 1$, write

$$A(z) = z^{-p-1}\big(A_0 + zA_1 + O(z^2)\big). \tag{30}$$

The package constructs exact nilpotent Jordan chains of A_0, uses A_1 to resolve the chain projector, and forms

$$T_Q(z) = \mathbf{1} - Q + \frac{Q}{z}, \qquad T_Q^{-1}(z) = \mathbf{1} - Q + zQ, \qquad Q^2 = Q. \tag{31}$$

For $I = T_Q J$ the complete transformed matrix is

$$A_{\text{new}} = T_Q^{-1} A T_Q - T_Q^{-1} T_Q'. \tag{32}$$

Every operation is performed over $\mathbb{Q}(i)$. A balance is retained only when exact idempotency holds and the pole order of the complete transformed matrix strictly decreases. The leading two Laurent matrices are then recomputed and the procedure is repeated until the system is Fuchsian. The manifest stores the ordered projectors and the complete pole-order history; applying all inverse balances to the final system must reproduce every original rational matrix entry exactly. A non-nilpotent highest Laurent matrix is an exact obstruction to a Fuchsian gauge in this unramified meromorphic class. Failure of an internal construction assumption is instead reported as inconclusive, not as genuine irregularity.

Local power–log solutions are evaluated in the reduced basis and multiplied by the complete product $T_{Q_1} \cdots T_{Q_m}$ before being returned. A singular-start $\{a, b, C\}$ record is validated directly in the original integral basis: FlintNDE generates an exact finite jet of the reduced Frobenius basis, multiplies the whole jet by the Laurent gauge, sets all earlier powers and higher logarithms to zero, and matches the requested coefficient C. This is necessary because a neighboring series coefficient can contribute to the original-basis leading vector under a non-diagonal balance.

If a higher pole remains, FlintNDE attempts a restricted exponential route. Suppose one exact constant basis over $\mathbb{Q}(i)$ makes all higher Laurent matrices diagonal and separates the system into sectors with identical diagonal high-pole signatures. For one such sector write

$$A_{\text{high}}(z) = \sum_{m=2}^{p} \frac{c_m}{z^m}\, \mathbf{1}. \tag{33}$$

With $Y = e^{\Phi(z)} U$, direct substitution gives

$$\Phi'(z) = \sum_{m=2}^{p} c_m z^{-m}, \qquad \Phi(z) = -\sum_{m=2}^{p} \frac{c_m}{m-1} z^{-(m-1)}. \tag{34}$$

The remaining equation for U must have pole order at most one, after which the ordinary power–log machinery applies unchanged. In particular, $A_{-2} = k$ gives $e^{-k/z}$. A computed $k = 0$ means only that this scalar exponential is absent; it does not remove a nilpotent, defective, or off-diagonal higher-pole constraint. Such a block must still pass Moser reduction or a more general formal reduction.

The exactly decoupled gate remains deliberately strict. It rejects a defective leading matrix, a spectrum outside $\mathbb{Q}(i)$, non-diagonal higher Laurent matrices in the selected basis, coupling between different exponential signatures, or a residual higher pole after subtraction. A boundary still uses $\{a, b, C\}$ for the power–log factor. The exact transformed C selects the exponential sector automatically; if one term spans several sectors the user must split it into separate terms.

There is one additional, explicitly formal route. For pole order exactly two, suppose A_{-2} has N simple distinct eigenvalues k_j in $\mathbb{Q}(i)$. For each right/left eigenvector pair (v_0, ℓ) FlintNDE constructs

$$Y = e^{-k/z} z^{\rho} \sum_{n=0}^{N_{\text{form}}} v_n z^n, \qquad \rho = \frac{\ell^T A_{-1} v_0}{\ell^T v_0}. \tag{35}$$

Equating the coefficient of $z^{\rho+q-2}$ gives

$$(k\mathbf{1} - A_{-2}) v_q = [A_{-1} - (\rho + q - 1)\mathbf{1}]\, v_{q-1} + \sum_{m=0}^{q-2} A_m v_{q-2-m}. \tag{36}$$

The left-eigenvector compatibility at order $q+1$ fixes the otherwise free v_0 component of v_q. All eigenvectors, ρ, compatibility tests, and recurrence solves are exact in $\mathbb{Q}(i)$; no numerical rank cutoff, least squares, or polynomial regression is used. The public boundary remains $\{a, b, C\}$: presently $b = 0$, $a = \rho$, and the exact vector C must select one leading eigendirection. The user does not enter k separately.

The occurrence of $e^{-k/z}$ does not by itself imply divergence. In the decoupled route, after the complete scalar high-pole part is removed, a residual system with at most a simple pole gives a convergent Frobenius power–log series whose disk is bounded by the nearest other singularity. By contrast, the recurrence in Eq. (36) is in general a sectorial Gevrey/asymptotic series. FlintNDE always evaluates through the requested degree N and generates five additional coefficient vectors for diagnostics; it does not silently replace N by the observed least-term degree. Define $T_n = v_n z^n$ and $S_N = \sum_{n=0}^{N} T_n$. The reported five-order block ratio and relative refinement are

$$r_5 = \frac{\left\| \sum_{j=1}^{5} T_{N+j} \right\|_\infty}{\left\| \sum_{j=0}^{4} T_{N-j} \right\|_\infty}, \qquad \delta_5 = \frac{\|S_{N+5} - S_N\|_\infty}{\|S_{N+5}\|_\infty}. \tag{37}$$

The five vector terms are summed before the infinity norm is taken. The desired decreasing-block condition is $r_5 < 1$. Failure does not suppress the result: FlintNDE emits a conspicuous warning and records the issue. The actual value is always reported, including when it is below one, so the user can judge whether the decrease is sufficient. The report also retains the least-term degree and magnitude as auxiliary information and says whether that degree was reached. For a caller that already owns scalar terms, the public `five_term_tail_diagnostic` instead evaluates

$$r_{5,\mathrm{scalar}} = \frac{\left| \sum_{j=1}^{5} T_{N+j} \right|}{\sum_{j=0}^{4} |T_{N-j}|}. \tag{38}$$

The returned `FiveTermTailDiagnostic` exposes the numerator, denominator, ratio, caller-supplied threshold, and strict pass flag. This scalar diagnostic does not replace Eq. (37); a zero denominator or an interval comparison that cannot certify the strict inequality fails closed. Generalized local expansions in Feynman-integral differential equations are discussed in Ref. [23].

Thus "how far from the irregular point may one evaluate?" has two different answers. The residual-Fuchsian case uses $|z| < R_{\mathrm{sing}}$ and the normal configurable step fraction. A formal asymptotic branch has no convergence radius. For automatic path generation FlintNDE uses the separation of the exponential roots. For sector i, define

$$\Delta_i = \min_{j \neq i} |k_i - k_j|, \qquad \widehat{n}_{\min,i}(z) \simeq \frac{\Delta_i}{|z|}. \tag{39}$$

This is the standard leading late-term estimate associated with the nearest competing exponential action. If N is the requested formal-series order and $q > 1$ is a safety factor, the proposed distance is

$$|z|_{\mathrm{root}} = \min_i \frac{\Delta_i}{qN}. \tag{40}$$

The default is $q = 3$, so N is approximately one third of the estimated least-term degree. For more than two roots, each branch uses its nearest other root and the smallest proposed distance is taken. A one-root scalar high-pole sector is handled by the exactly decoupled exponential power–log route rather than this formal root-gap rule. The path interval and `max_step_over_radius` R may only reduce the distance; the latter is applied only when it gives a smaller step. Final acceptance is nevertheless based on the actual tests in Eq. (37); the root-gap estimate is not a proof of convergence or accuracy.

The distance to the nearest other singularity is therefore only a reference scale, not a convergence radius. The user must remain in one Stokes sector and verify order and matching-point

stability. User-supplied first matching points receive the same five-order report and warning. The formal route is therefore allowed only as the path start. An internal or target use is rejected because FlintNDE has no Stokes matrix with which to connect the two sides.

AMFlow provides a concrete engineering reference for the reducible part of this problem [4, 5]. Its DE solver first decomposes the matrix into blocks. On each diagonal block it iteratively constructs a projector from the leading two local coefficients and applies a balance transformation until the Poincaré rank vanishes; it then uses shearing transformations to normalize residue eigenvalues. The remaining off-diagonal high-order blocks are reduced by solving linear matrix equations, after which the transformed boundary is passed to the singular Frobenius solver. If the leading matrix has a nonzero Jordan eigenvalue, the projector route declares the positive Poincaré rank irreducible and aborts. FlintNDE now implements the exact local projector-balance iteration on the complete matrix, including ordered projectors and a full rational-system round trip. It does not reproduce AMFlow's global block decomposition, residue normalization, or separate off-diagonal block solver; those are normalization and global-organization layers beyond the local question of whether the supplied pole can be reduced to Fuchsian form.

More general irregular systems require sectors of the form

$$Y(z) = e^{\Phi(z)} z^{\rho} \sum_{n,q} v_{n,q} z^{n} (\log z)^{q}, \qquad \Phi(z) = \sum_{j \geq 1} \phi_j z^{-j}. \tag{41}$$

The implemented convergent scalar-sector case is Eq. (34); the start-only simple-spectrum case is Eq. (36). Repeated/defective leading roots, higher Poincaré rank, ramification, algebraic field extensions, general formal block gauges, and Stokes matrices are not implemented. The dispatcher fails closed whenever those data would be needed; it never accepts "set $k = 0$" as a substitute for the missing reduction.

6.5 Numerical reconstruction in a regulator

Many dimensionally regulated calculations require coefficients of

$$F(\epsilon) = \sum_{n=n_{\min}}^{n_{\max}} f_n \epsilon^n + O(\epsilon^{n_{\max}+1}). \tag{42}$$

Instead of truncating every intermediate expression in ϵ, one may solve the complete numerical problem at several small exact-rational values ϵ_j and reconstruct the final Laurent coefficients. AMFlow uses this strategy [4].

If the target coefficient has order m, $N{+}1$ samples lie at radius r, and the nearest unaccounted singularity is at scale R, the estimate quoted in the AMFlow analysis is

$$E_n \sim \left(\frac{r}{R}\right)^{N+1-n}, \qquad N \sim m - 1 + \frac{\log E}{\log(r/R)}. \tag{43}$$

If a single solve to error p costs $t(p) \sim (-\log p)^{\alpha}$, the reference optimization is

$$r \sim R E^{1/(\alpha m)}, \qquad N \sim (\alpha+1)m - 1, \qquad p \lesssim E^{(\alpha+1)/\alpha}. \tag{44}$$

These are guidance formulas, not a posteriori proofs.

AMFlow 2.0 uses an empirical configuration [5] in which L is the number of loops and the assumed leading Laurent power is $n_{\min} = -2L$. Its argument called "order" is the reconstruction

depth K measured upward from $n_{\min}$, not the absolute highest power. For decimal goal g,

$$N_{sample} = \left\lceil \frac{5}{2}K + 2L \right\rceil, \tag{45}$$

$$\alpha_\epsilon = \frac{L}{2} + \frac{g}{K+1}, \tag{46}$$

$$\epsilon_0 \simeq 10^{-\alpha_\epsilon}, \qquad \epsilon_j = \epsilon_0 \left(1 + \frac{j}{100}\right), \quad j = 1, \ldots, N_{sample}, \tag{47}$$

$$p_0 = \max\left(\left\lceil (N_{sample} + 2L)\alpha_\epsilon \right\rceil, 30\right), \tag{48}$$

$$p_{work} = \left\lceil 2(1 + p_{extra})p_0 \right\rceil. \tag{49}$$

The corresponding solver settings are $X_{\text{order}} = 4p_0$ and $X_{\text{extra}} = \lceil \max(50, p_0/5) \rceil$. The one-percent spacing, rationalization of sample points, doubled working precision, and these two expansion orders are implementation choices in AMFlow 2.0 rather than consequences of Eqs. (43) and (44). FlintNDE implements the automatic choice as $\max(200, \lceil 2(1 + p_{extra})p_0 \rceil)$. The 200-digit lower bound is a FlintNDE default rather than part of the quoted AMFlow formula; an explicit `working_precision_digits` value overrides automatic planning.

The FlintNDE interface should not require a loop count: L is specific to the Feynman integral pole bound used by AMFlow and is not an intrinsic parameter of a general matrix differential equation. Nor can a finite collection of numerical regulator solves prove the Laurent support of arbitrary Python callables. The current interface therefore requires an integer $n_{\min}$ together with a strict upstream certificate whose status is certified, whose recorded integer agrees with $n_{\min}$, and whose method identifies the symbolic boundary-and-DE analysis. Missing certificates, automatic leading powers, and inconsistent integers fail before any NDE solve. MadStree produces this certificate automatically from its physical boundary and dlog DE; FlintNDE remains independent of those physical data structures.

Let the certified global leading power be $n_{\min}$, let the user request the absolute highest power m, and define

$$P = \max(0, -n_{\min}), \qquad K = m - n_{\min}. \tag{50}$$

The automatic production plan uses the AMFlow 2.0 formula after the transparent replacements $2L \to P$ and $L/2 \to P/4$:

$$N_{\text{fit}} = \max\left(\left\lceil \frac{5}{2}K + P \right\rceil, K + 1\right), \tag{51}$$

$$\alpha_\epsilon = \frac{P}{4} + \frac{g}{K+1}, \qquad \epsilon_0 \simeq 10^{-\alpha_\epsilon}, \tag{52}$$

$$p_0 = \max\left(\left\lceil (N_{\text{fit}} + P)\alpha_\epsilon \right\rceil, 30\right), \qquad p_{work} = \left\lceil 2(1 + p_{extra})p_0 \right\rceil. \tag{53}$$

All N_{fit} production samples are used in a square interpolation from $n_{\min}$ through the internal power $n_{\min} + N_{\text{fit}} - 1$; only coefficients through the requested power m are returned. Thus the powers above m absorb truncation effects exactly as in the AMFlow construction; the extra production points are neither discarded nor converted into a least-squares fit. At least N_{check} additional points, not used in that interpolation, validate the retained coefficients. All component coefficients are determined by the same square Acb interpolation and are tested at the independent points. The reported result includes the upstream certificate, accepted leading power, all effective parameters, the internal and returned power ranges, coefficient vectors, and independent residuals.

The implementation initially enforces at least `fit_extra_order=2` powers above m. If the independent residual is too large, each round adds `fit_order_increment=2` production points and corresponding powers. Only those new points are solved; prior production values and the separately cached validation values are reused. The default `fit_max_rounds=3` then fails closed

at the unchanged tolerance. This incremental a posteriori check supplements the planning estimates above.

For example, if the upstream certificate gives $n_{\min} = -4$, while the user requests $m = 2$ and $g = 30$, then $P = 4$, $K = 6$, and

$$N_{fit} = 19, \qquad \alpha_\epsilon = \frac{37}{7} \simeq 5.285714, \qquad \epsilon_0 \simeq 5.18 \times 10^{-6}. \tag{54}$$

The interpolation internally covers powers $-4, \ldots, 14$ and returns only $-4, \ldots, 2$. The same defaults give $p_0 = 122$, $p_{work} = 244$ decimal digits, a primary transport order $4p_0 = 488$, and an extra transport order $\lceil \max(50, p_0/5) \rceil = 50$. With the default two validation points, this stage performs 19 production solves and two independent validation solves.

6.5.1 Power-series-solution interface

The implemented high-level command is

```
series_result = reconstruct_series_solution(
    DEmatrix=DEmatrix,
    boundary=boundary,
    path=path,
    maximum_power=2,
    leading_power=-1,
    leading_power_certificate={
        "status": "certified",
        "leading_power": -1,
        "method": "symbolic-boundary-and-DE",
    },
)
```

It calls the same basic NDE solver independently at every regulator sample. All algorithm controls are optional keyword overrides:

```
series_result = reconstruct_series_solution(
    DEmatrix=DEmatrix,
    boundary=boundary,
    path=path,
    maximum_power=2,                # required target
    leading_power=-1,               # required certified support
    leading_power_certificate={
        "status": "certified",
        "leading_power": -1,
        "method": "symbolic-boundary-and-DE",
    },
    series_parameter="ep",
    goal_digits=30,
    sample_points="automatic",
    sample_count="automatic",
    base_sample="automatic",
    sample_spacing=0.01,
    working_precision_digits="automatic",
    extra_working_precision=0.0,
    transport_order="automatic",
    transport_extra_order="automatic",
```

```
    transport_sample_count="automatic",
    transport_extra_sample_count="automatic",
    radius_fraction=0.60,
    guard_bits=32,
    validation_sample_count=2,
    validation_scale=0.5,
    validation_tolerance="automatic",
    maximum_samples=100,
    rationalize_sample_points=True,
    fit_extra_order=2,
    fit_order_increment=2,
    fit_max_rounds=3,
    parallel_task_count=12,
    output_layout=None,
    result_name="series_reconstruction",
)
```

Here `transport_order` is the primary local-series order of the underlying NDE solve, and `transport_extra_order` is the added order used for its refinement check. Their automatic values are $4p_0$ and $\lceil \max(50, p_0/5) \rceil$, respectively. The literal `"automatic"` selects documented production controls only; a user value replaces only that decision. No public object is named after AMFlow: the citation records the origin of the default empirical formulas, while the package feature is power-series-solution reconstruction.

Each of the first three inputs may be fixed, or regulator dependent with the signatures `DEmatrix(ep)`, `boundary(ep)`, and `path(ep, system)`, respectively. With rationalization enabled, generated real samples are passed to these factories as FLINT `fmpq` values, so a sample-dependent exact rational matrix can still use the exact singularity and Frobenius gates. A fixed system is an `AnalyticMatrixSystem` or `RationalMatrixSystem`; a fixed ordinary boundary is an Acb column matrix or scalar list; and a fixed path is a direct `list[acb]`. A singular start instead uses the exact $\{a, b, C\}$ protocol in Sec. 6.6.4; every regulator sample reuses the same local reduction, indicial, and log compatibility checks before transport begins.

6.6 Complete public interface and parameter reference

The user environment requires only Python 3.10 or later and `python-flint>=0.6`. After publication the package can be installed from the package index with

```
python -m pip install FlintNDE
```

or from a downloaded wheel with

```
python -m pip install ./flintnde-0.4.0-py3-none-any.whl
```

A downloaded source checkout can be installed normally with

```
python -m pip install path/to/package-FlintNDE/versions/FlintNDE-0.4.0
```

and developers may instead select editable mode with

```
python -m pip install -e path/to/package-FlintNDE/versions/FlintNDE-0.4.0
```

Neither the wheel nor source-install route requires the user to build the package manually; `pip` installs the declared `python-flint` dependency automatically. All public names are imported from `flintnde`. An exact scalar accepts an integer, a FLINT `fmpq`, a rational or decimal string, a string such as `"a+b*I"`, a two-tuple `(real, imag)`, or a dictionary with `"real"`

and "imag". The public gaussian_rational converter returns a GaussianRational in $\mathbb{Q}(i)$. Python float, complex, arb, and acb values are rejected by the exact route; they belong to an explicitly numerical system with a stated reliable input precision. The recommended build_frobenius_manifest dispatcher follows two rules:

- RegularSingularSystem selects exact classification, and exact failure is returned directly;
- NumericalRegularSingularSystem selects numerical classification for data explicitly supplied as floating point.

6.6.1 Caller-local output layout

The package does not write into its installation tree or infer an output location from the process working directory. Every calling script can initialize a stable sibling layout:

```
layout = initialize_output_layout(__file__, run_name=None)
summary_path = layout.write_json(
    "summary", "result_summary.json", summary,
)
```

After one installation, this script may live in any directory A and import flintnde without copying the package source. Passing that script's __file__ places managed outputs under A/results/<run_name>/. A JSON, TOML, or other configuration file may live beside the script and be read by it, but the current release does not provide a configuration-only command-line runner: a Python caller is still required to invoke the public API.

The default run name is the caller filename without its extension. The resulting tree is

```
calling_script.py
results/<run_name>/
    configuration/output_layout.json
    singularities/
    frobenius/
    transport/
    regularization/
    summary/
```

Only the configuration directory and its layout manifest are created eagerly. The other categories appear when first used. Exact finite/infinite inventories belong in singularities; Frobenius classification and power–log metadata belong in frobenius; path, segment, and connection data in transport; sampling plans and Laurent coefficients in regularization; and final scalars and checks in summary.

Table 2: Caller-local output parameters.

Interface or parameter	Meaning and constraints
initialize_output_layout	Parameters are caller_script and keyword-only run_name=None. The former is an existing script path, normally __file__; the latter defaults to its stem and, when supplied, must be one nonempty path component. It returns an OutputLayout rooted at <caller>/results/<run_name>/.
OutputLayout. directory(category)	category is exactly one of configuration, singularities, frobenius, transport, regularization, or summary. The directory is created on demand.

Interface or parameter	Meaning and constraints
`OutputLayout.file(category, filename)`	`filename` must be one nonempty relative path component, not an absolute or nested path. The method creates the category directory and returns the file path without creating the file.
`OutputLayout.write_json`	Parameters are `category`, `filename`, and JSON-serializable `payload`. It writes UTF-8 JSON with stable indentation and returns the written path.
`OutputLayout` fields	`caller_script`, `run_name`, `results_root`, and `run_root` are resolved read-only paths/metadata. Users normally obtain the object from `initialize_output_layout` rather than constructing it directly.

6.6.2 Exact Gaussian-rational singularities and unified dispatch

Polynomial coefficients are always ordered from the constant term upward. The following input contains both a nonreal coefficient and nonreal poles:

```
layout = initialize_output_layout(__file__, run_name="point_01")
system = RationalMatrixSystem(
    (
        (rational_function([0, "1+I"], ["-2-3*I", 1]), 0),
        (0, rational_function(
            {"real": "1/3", "imag": "-2/5"}, [1, 0, 1]
        )),
    ),
    variable_name="s", name="example-system",
)
inventory = analyze_singularities(system, output_layout=layout)
path = build_adaptive_path(
    system,
    NamedPoint("left_boundary", 2),
    NamedPoint("right_match", 3),
    path_name="left_to_right",
    max_step_over_radius=0.45,
    output_layout=layout,
)
local = prepare_local_expansion(
    system, NamedPoint("left_boundary", 2),
    order=32, radius_fraction=0.60, sample_count=96,
    output_layout=layout,
)
snapshots, segment_reports, elapsed = transport_path(
    system, column_vector([1, 0]), path,
    order=32, sample_count=96, radius_fraction=0.60,
)
```

Integers, FLINT `fmpq` values, rational or decimal strings, `"a+b*I"`, real/imaginary tuples, and real/imaginary dictionaries are exact coefficient inputs. Python `float`, `complex`, `arb`, and Acb values are rejected by automatic denominator discovery. A sum of rational terms should be constructed with ordinary `+`; the resulting complete entry is reduced before factor collection. A black-box `AnalyticMatrixSystem` cannot expose all poles from finitely many samples and must continue to declare its singularities explicitly. Because a top-level tuple in

`rational_function((c0,c1))` denotes polynomial coefficients, a single real/imaginary tuple is wrapped in a one-element coefficient list or converted first with `gaussian_rational`; this removes the only syntactic ambiguity. Roots that lie in $\mathbb{Q}(i)$ retain an exact location and can be dispatched directly to the exact Frobenius constructor. More general algebraic roots are still isolated in the inventory as Acb balls, but exact Frobenius analysis over a general algebraic number field is not implemented. Likewise, an indicial polynomial that does not split completely over $\mathbb{Q}(i)$ fails closed instead of falling back to a threshold decision.

Table 3: Exact singularity, local dispatch, and named-path parameters.

Interface or parameter	Meaning and constraints
`rational_function`	Parameters are `numerator` and `denominator=(1,)`. A scalar is promoted to one coefficient; a list or tuple is ordered by increasing power. The return value is an exact reduced `RationalFunction`.
`gaussian_rational`	Parameters are `value=0` and optional `imag=None`. It validates and converts the exact scalar forms listed above to `GaussianRational`; floating inputs raise `TypeError`.
`GaussianRational`	Read-only `real` and `imag` fields are FLINT `fmpq` values. Exact arithmetic, conjugation, Acb conversion, and certified square roots within $\mathbb{Q}(i)$ are provided; roots outside this field fail closed.
`RationalFunction`	Constructor fields are coefficient tuples `numerator` and `denominator=(1,)`. Addition combines terms exactly. Methods `evaluate(point)`, `exact_polynomials()`, `inverted_variable()`, and `pole_order_at_infinity()` expose the reduced evaluation and infinity transformation.
`RationalMatrixSystem`	Constructor parameters are square nested `entries`, `variable_name="s"`, and `name`. Entries may be `RationalFunction`, an exact constant, or a dictionary with `numerator` and optional `denominator`. Fields are immutable after coercion.
`singularity_inventory`	Keyword-only `root_tolerance=1e-30`. It exact factors every reduced denominator, isolates all finite complex roots, and separately classifies infinity. The return value is `SingularityInventory`.
`analyze_singularities`	Parameters are `system`; keyword-only `root_tolerance=1e-30`, `output_layout=None`, and `filename="singularity_inventory.json"`. When a layout is supplied, all locations, pole orders, factors, and witness matrix entries are saved under `singularities`.
`RationalMatrixSystem.inverted`	Keyword `variable_name="sinv"`; returns the exact matrix in Eq. (17).
`NamedPoint`	Constructor fields are `name` and `value`. A finite value accepts the usual Acb-compatible scalar forms. Infinity is accepted only as the literal string `"inf"`.
`classify_point`	Parameters are a `NamedPoint` and `SingularityInventory`. It returns a `PointClassification` containing the kind, pole order, matched singularity identifier, and convergence radius.

Interface or parameter	Meaning and constraints
`prepare_local_expansion`	Parameters are `system`, `endpoint`; keyword-only `order`, `radius_fraction=0.60`, `radius=None`, `sample_count=None`, and `output_layout=None`. It selects `ordinary_cauchy_dft`, `regular_singular_power_log`, `fuchsian_reduced_power_log`, `exponential_power_log`, or the start-only `formal_exponential_asymptotic`. An unbounded ordinary disk requires explicit `radius`; a formal basis reports no convergence radius; an uncertified high-pole point raises `LocalReductionError`.
`LocalExpansion`	Read-only fields are `classification`, `working_variable`, `method`, `convergence_radius`, optional `matrix_coefficients`, optional `power_log_basis`, and `manifest`.
`attempt_fuchsian_reduction`	Parameters are an exact `RationalMatrixSystem` and a $\mathbb{Q}(i)$ point. It iterates the exact Lee–Moser balance in Eq. (31). It returns the ordered projectors, original and reduced pole orders, complete pole-order history, status, reason, and an exact round-trip-certified transformation object.
`build_local_solution_basis`	Parameters are an exact system, point, and positive series order. It dispatches regular power–log, sheared power–log, certified exponential-times-power–log, or the simple-spectrum rank-one formal recurrence and returns a `LocalSolutionBasis` in the original integral basis. Its `continuation_ready` flag is false for the formal route; `evaluation_report` returns the fixed order, five-order block ratio and refinement, plus auxiliary least-term diagnostics. Unsupported formal/Stokes problems raise `LocalReductionError`.
`five_term_tail_diagnostic`	Parameters are scalar evaluated terms through degree $N+5$, the evaluation degree $N \geq 4$, and an optional positive threshold. It returns a `FiveTermTailDiagnostic` implementing Eq. (38); equality does not pass.
`build_adaptive_path`	Parameters are `system`, `start`, and `target`; keyword-only `detour_points=()`, `path_name=None`, `max_step_over_radius=0.45`, `path_tolerance=1e-24`, `formal_asymptotic_order=None`, `formal_minimum_order_factor=3.0`, and `output_layout=None`. A positive formal order activates the root-gap formal-start estimate; the factor must exceed one. It returns an `AdaptivePath`, which is a `list[acb]` accepted unchanged as the `path` argument of `transport_path`. Every reported step satisfies $\lvert\Delta z\rvert/R \leq$ `max_step_over_radius`. A connection-ready internal pole emits a warning and receives a local bridge; a start-only formal point in the interior or at the target is rejected because Stokes data are absent. Finite ordinary `detour_points` remain optional.
`AdaptivePath`	Besides normal list operations, read-only diagnostics are `step_reports`, `internal_singularities`, `start_classification`, `target_classification`, `working_variable`, `infinity_transformation`, `formal_asymptotic_match_estimate`, and `max_step_over_convergence_radius`. The ordered reports contain step length, controlling radius, radius owner, ratio, method, and singularity identifier. A step into a singularity always names the singularity as its radius owner.

Interface or parameter	Meaning and constraints
`build_adaptive_path_plan`	Diagnostic interface using the same named endpoints, step ratio, path tolerance, `local_analysis_order=4`, and output layout. It returns metadata rather than the numerical path argument and preflights every high-pole checkpoint.
`PathPlan`	Read-only fields include both variable names, infinity-transform flag, endpoint classifications, all internal singularities, generated named points, segment methods, step/radius ratios, messages, and `continuation_ready`. Regular singularities and high-pole points with a certified shearing or convergent exponential local basis are ready. A formal asymptotic point is ready only when it is the start; at an internal point or target the flag is false and the missing Stokes connection is recorded.
`SingularityRecord`	Stores identifier, Acb location, optional exact linear root, pole order, classification, exact reduced-denominator factor, and zero-based witness matrix entries.
`SingularityInventory`	Stores all finite records, the infinity record, system name, variable name, and a JSON serializer.

6.6.3 Ordinary-point workflow

A minimal ordinary-point calculation is

```
from flint import acb, acb_mat
from flintnde import *

# The import has already selected the 200-digit default.
system = AnalyticMatrixSystem(
    matrix_function=lambda z: acb_mat([[0, 1], [-1/(1-z), 0]]),
    dimension=2, singularities=(acb(1),), name="example",
)
path = build_straight_path(system, acb("0.1"), acb("0.7"))
result = transport_path_refined(
    system, column_vector([1, 0]), path,
    primary_order=24, reference_order=30,
)
```

The callback must return an `acb_mat` of the declared dimension. Every singularity that can limit a Cauchy disk must be listed. An empty list disables automatic radius and path control rather than asserting that the matrix is entire.

Table 4: Precision, matrix-system, and transport parameters.

Interface or parameter	Meaning and constraints
`configure_working_precision`	Parameters are `decimal_digits` and `guard_bits=32`. It sets the process-wide FLINT context to $\lceil \texttt{decimal\_digits} \log_2 10 \rceil + \texttt{guard\_bits}$ bits and returns that bit count. Decimal digits must be positive and guard bits nonnegative.

Interface or parameter	Meaning and constraints
`AnalyticMatrixSystem`	Constructor parameters are `matrix_function`, `dimension`, `singularities=()`, and `name`. The callback `matrix_function(z)` returns the coefficient matrix; `dimension` is its square size; `singularities` supplies Acb locations for radius control; and `name` labels errors and reports.
`evaluate(point)`	Evaluates the callback and checks the declared dimension.
`nearest_singularity_distance`	Its `point` parameter is an Acb value. It returns the minimum declared distance and raises an error when no singularities were supplied.
`taylor_matrix_coefficients`	Parameters are `center`, `order`, keyword-only `radius`, and `sample_count=None`. It reconstructs $A_0, \ldots, A_{\texttt{order}-1}$. `radius` is nonzero and must lie in the analytic domain. The default sample count is $\max(32, 2\,\texttt{order})$ and any supplied count must exceed `order`.
`build_straight_path`	Parameters are `system`, `start`, `target`, and keyword-only `step_fraction=0.2`. It builds a straight Acb path. Each step is at most `step_fraction` times the nearest singularity distance; the fraction lies strictly in $(0, 1)$. A path that must avoid a pole should instead be supplied with complex detour points.
`transport_path`	Parameters are `system`, `initial_vector`, `path`, and keyword-only `order`, `sample_count=None`, `radius_fraction=0.6`, and `target_relative_error=None`. It transports one Acb column along at least two path points. At an exact singular start, `initial_vector` is instead the `FrobeniusBoundary` of Sec. 6.6.4; the name is retained for API compatibility. `order` is the Taylor order; `sample_count` follows the Cauchy rule above; and `radius_fraction` $\in (0, 1)$. The `list[acb]` returned by `build_adaptive_path` is accepted directly. Pass the original `RationalMatrixSystem` when the path may contain a singular local bridge; ordinary lists may still use `AnalyticMatrixSystem`. Normally `radius_fraction` should exceed `max_step_over_radius`; each ordinary step must stay inside its sampling circle. A bridge report records its power-log order, maximum log degree, branch convention, the two step/radius ratios, and elapsed time. At a formal start, the optional accuracy target is compared with δ_5 and $r_5 < 1$ is always checked; failures warn but do not remove the result. The function returns `(snapshots, segment_reports, elapsed_seconds)`.
`transport_path_refined`	In addition to `system`, `initial_vector`, and `path`, keyword-only parameters are `primary_order`, `reference_order`, `primary_sample_count=None`, `reference_sample_count=None`, `radius_fraction=0.6`, and `target_relative_error=None`. It runs independent primary and reference chains. The reference order must exceed the primary order. The returned dictionary contains both chains, timings, segment reports, the final infinity-norm relative difference, and `target_relative_error_met`. A failed target emits a warning but preserves both chains.

Interface or parameter	Meaning and constraints
`transport_frobenius_boundaries_refined`	Parameters are an exact `RationalMatrixSystem`, a nonempty list of `FrobeniusBoundary` objects, and one common path, followed by `primary_order`, `reference_order`, `radius_fraction=0.6`, and `target_relative_error=None`. Within each order it constructs the local basis once, packs the independently supplied boundaries as matrix columns, and reuses each ordinary-segment Taylor matrix. The return dictionary contains matrix snapshots, shared segment reports, per-column boundary reports, `relative_differences_inf`, and per-column `target_relative_error_met`. The current batch route requires every transition after the singular launch to be `ordinary_taylor` and rejects coordinate `save` tags. This is only a batch-optimization restriction: the ordinary `transport_path` and `transport_path_refined` interfaces save a direct regular-singular start or target as reusable `frobenius_boundary` data containing the verified (a, b, C) terms. Callers use separate `transport_path_refined` calls when saved ordinary points or singular boundary constants are requested.
`column_vector(values)`	Converts a list of Acb or exact real scalars to an Acb column matrix.
`acb_midpoint_matrix(matrix)`	Rebuilds a matrix from high-precision midpoints; this limits wrapping but discards enclosure radii.
`vector_norm_inf(vector)`	Returns the largest entry magnitude of a column.
`relative_difference_inf`	Parameters are equal-dimensional `primary` and `reference` columns. It returns $\|\texttt{reference} - \texttt{primary}\|_\infty / \|\texttt{reference}\|_\infty$ for equal dimensions and rejects a reference norm whose ball contains zero.

6.6.4 Regular-singular workflow

For exact $\mathbb{Q}(i)$ data, the preferred workflow is

```
system = RegularSingularSystem(
    residue_exact=[["I", "0"], ["0", "1+I"]],
    regular_coefficients_exact=([["0", "0"], ["1", "0"]],),
    name="resonant-example",
)
manifest = build_frobenius_manifest(system)
basis = build_power_log_basis(system, manifest, series_order=20)
fundamental_matrix = basis.evaluate(acb("0.01"))
```

The tuple `regular_coefficients_exact` is ordered as $(A_0, A_1, \ldots)$; missing higher coefficients are zero. The manifest contains the roots and multiplicities, semisimplicity data, every resonance gate, and the actual maximum log degree activated by those gates.

For numerical data, use

```
system = NumericalRegularSingularSystem(
    residue, regular_coefficients, name,
    input_precision_digits=80,
)
options = NumericalFrobeniusOptions(relative_zero_tolerance=None)
manifest = build_frobenius_manifest(system, options)
```

The warning is unconditional, including with a user tolerance. The `numeric_diagnostics` block records `matrix_scale`, both cutoffs, and the zeroed count. Separate largest-zeroed and smallest-retained fields give absolute and relative-to-matrix-scale values.

Boundary data at a singular start. A singular point does not in general carry a finite value of Y. The boundary input therefore specifies one or more leading power–log factors,

$$\{a_i, b_i, C_i\} \quad \longleftrightarrow \quad z^{a_i}(\log z)^{b_i} C_i, \tag{55}$$

where C_i is the complete leading coefficient vector of all master integrals, including its zero components. Here $z = s - s_0$ at a finite start s_0, while $z = \texttt{sinv} = 1/s$ at infinity. Thus a_i is always an exponent in the actual local variable used by the local system.

The executable input is

```
boundary = frobenius_boundary([
    {"a": "0", "b": 1, "C": [1, 0]},
    {"a": "0", "b": 0, "C": [2, 0]},
])

path = build_adaptive_path(
    DEmatrix,
    NamedPoint("start", 0),
    NamedPoint("target", 1),
)
snapshots, reports, elapsed = transport_path(
    DEmatrix, boundary, path, order=24,
)
```

Each record may equivalently be written as the tuple `(a, b, C)`. The entries `a` and `C` must be exact $\mathbb{Q}(i)$ inputs; Python floating-point, complex, Arb, and Acb values are rejected. The log degree `b` must be a nonnegative Python integer, and the nonzero vector `C` must have the differential-equation dimension. A finite Acb column is still required at an ordinary start. Supplying that column at a singular start, or supplying a Frobenius record at an ordinary start, fails before the first transport segment.

For a semisimple residue, the gate requires $b_i = 0$ and $C_i \in \ker(R - a_i \mathbf{1})$. For a single-root Jordan sector $R = a\mathbf{1} + N$, a record of degree b determines canonical basis constants c through

$$N^b c = b! C, \qquad N^{b+1} c = 0. \tag{56}$$

The package solves this exact linear system by RREF with free variables set to zero. It then generates all lower log powers from $z^a \exp(N \log z)c$; users do not repeat those coefficients in the input. Multiple records are superposed. Exact indicial membership, eigenspace or Jordan-chain compatibility, the maximum log degree, and a nonzero combined solution are all checked before numerical evaluation. Resonance logarithms at higher series orders remain properties of the already verified power–log basis and are generated automatically.

For a continuation-ready irregular sector the reusable record is

$$\{\phi_i, a_i, b_i, C_i\} \quad \longleftrightarrow \quad \exp\big(\phi_i(z)\big) z^{a_i} (\log z)^{b_i} C_i, \qquad \phi_i(z) = \sum_{p<0} \phi_{i,p} z^p. \tag{57}$$

Every p is a negative Python integer and every $\phi_{i,p}$ is exact in $\mathbb{Q}(i)$. The executable form is

```
boundary = exponential_boundary([{
    "phi": [
```

```
        {"power": -2, "coefficient": "3/2"},
        {"power": -1, "coefficient": "-1"},
    ],
    "a": "2/3", "b": 0, "C": [1, 0],
}])
```

Repeated powers are combined and exact zero coefficients removed. An empty `phi` list means $\phi = 0$ and $\exp(\phi) = 1$; this keeps a zero-signature sector in the same reusable format as other sectors of one high-pole system. The record does not let the user choose a new exponential: FlintNDE derives each signature from the high-order Laurent matrices and requires exact equality. Exponential input must therefore provide $\{\phi, a, b, C\}$, and saved output records the same complete schema.

For a Moser-reduced high-pole start, the same record is interpreted in the original integral basis. The package multiplies an exact reduced-basis power–log jet by the complete Laurent gauge and solves the earlier-power, higher-log, and requested-C_i constraints in that basis; it does not transform only the isolated leading vector. For an exponential start, the exact constant-basis transform of C_i identifies the sector, while the required input ϕ is checked against the exponential of Eq. (34). One term must not mix distinct exponential signatures. In either case, splitting a physical boundary into several records represents the required linear superposition.

After validation, the local basis is evaluated at the first ordinary matching point of the adaptive path and ordinary transport continues from that value. The first segment report names the actual regular, Moser-reduced, or exponential initialization route; it records the original terms, reduction manifest, canonical basis constants, matching point, working variable, and principal Acb logarithm/power branch. The same protocol is used independently at every production and validation sample of `reconstruct_series_solution`. A numerical Frobenius manifest and the pre-existing mixed-root defective exact case remain fail closed because neither currently supplies a certified canonical chain basis for this boundary conversion.

Table 5: Singular local-basis and power–log parameters.

Interface or parameter	Meaning and constraints
`RegularSingularSystem`	Constructor parameters are `residue_exact`, `regular_coefficients_exact`, and `name`. The residue is R; `regular_coefficients_exact` is the tuple of A_m matrices; `name` is the manifest identity. All matrices have one dimension and exact $\mathbb{Q}(i)$ entries.
`frobenius_boundary`	Its `records` parameter is a nonempty iterable of exact mappings whose fields are precisely `a`, `b`, and `C`, or of `(a,b,C)` tuples. It validates the record schema and exact scalar field immediately and returns a frozen `FrobeniusBoundary`; DE-dependent indicial and log checks occur at transport.
`exponential_boundary`	Its `records` parameter is a nonempty iterable of exact mappings with fields `phi`, `a`, `b`, and `C`. `phi` is a possibly empty list of negative-integer `power` and exact `coefficient` pairs. It returns `ExponentialBoundary`; the concrete local basis later checks the complete signature against the inferred exponential sector.
`FrobeniusBoundaryTerm`	Constructor fields are `a`, `b`, and `C`. They store one exact highest-log leading term in Eq. (55). Direct construction applies the same validation as the factory.
`FrobeniusBoundary`	Its `terms` field is a nonempty tuple of `FrobeniusBoundaryTerm` objects with one common vector dimension. Users normally construct it through `frobenius_boundary`.

Interface or parameter	Meaning and constraints
`build_frobenius_manifest`	Parameters are `system` and optional `options=None`. A `RegularSingularSystem` always selects the exact gate and rejects numerical options; exact failure is returned directly. A `NumericalRegularSingularSystem` selects the numerical gate and may receive `NumericalFrobeniusOptions`.
`build_exact_frobenius_manifest`	Its sole parameter is `system`. It is the explicit low-level exact entry and performs the original-dimension $\mathbb{Q}(i)$ indicial, Jordan, projector, and resonance decision. The indicial polynomial must split completely over $\mathbb{Q}(i)$; unsupported mixed-root defective structure fails closed.
`exact_data`; `acb_data`	`exact_data()` takes no parameter; `acb_data(order)` requires positive `order`. They return the validated exact matrices or their Acb conversion. Positive `order` controls how many regular coefficients are returned, with missing A_m filled by zero matrices.
`NumericalRegularSingularSystem`	Constructor parameters are `residue`, `regular_coefficients`, `name`, and `input_precision_digits=None`. It accepts Acb-convertible floating values, including `acb`, `arb`, complex values, real/imaginary pairs, and decimal strings. The last parameter records the reliable decimal precision when it cannot be inferred from a machine float or ball.
`acb_data` (numerical system)	Takes no parameter and returns the validated Acb residue and the supplied regular-coefficient list without adding higher zero coefficients.
`NumericalFrobeniusOptions`	Constructor parameters are `precision_digits=None` and `relative_zero_tolerance=None`. The former declares reliable input digits. Python binary64 input imposes a 15-digit upper bound even when a larger value is supplied. If neither the input nor the system/options declare a precision, classification fails closed. The default relative cutoff is $N10^{-p}$. A positive decimal string supplied as `relative_zero_tolerance` overrides it.
`build_numerical_frobenius_manifest`	Parameters are `system` and `options=None`. It is the explicit low-level Acb midpoint and threshold-dependent entry. Omitting `options` still requires precision information from the numerical system or its machine/ball inputs. Uncertain cases fail closed.
`build_power_log_basis`	Parameters are `system`, `manifest`, and keyword-only `series_order`. It reuses the exact or threshold-gated structure and computes coefficients through positive `series_order`. The system name and manifest status must match.
`PowerLogBasis.evaluate`	Its `point` parameter is a nonzero Acb value. It evaluates the fundamental matrix there, using the FLINT logarithm branch selected by that point.
`PowerLogBasis.dimension`, `maximum_log_degree`, `manifest`	Read-only result fields. Users obtain this object from `build_power_log_basis`; its private evaluator constructor argument is not a user option.

6.6.5 Power-series-solution reconstruction

The public entry is the single high-level command shown in Sec. 6.5. It reuses the basic NDE transport internally and does not expose a separate sampling-plan class or a separate fitting command. The minimal call supplies only the differential equation, boundary, path, and desired highest regulator power; the leading power and all numerical controls are automatic unless

overridden.

Table 6: Power-series-solution reconstruction parameters.

Interface or parameter	Meaning and constraints
`DEmatrix`, `boundary`, `path`	Required inputs passed to the same basic NDE solver at every regulator value. They may be fixed objects or factories with signatures `DEmatrix(ep)`, `boundary(ep)`, and `path(ep, system)`. The path has the same direct list format as an ordinary NDE call. The boundary is an ordinary-point column vector or, when the path starts at an exact regular singularity, the `FrobeniusBoundary` of Sec. 6.6.4. A boundary factory may return either format at each regulator sample; incompatible formats fail closed.
`maximum_power`	Required absolute highest regulator power m returned to the user. There is no public `series_range`; the lowest power is supplied separately.
`leading_power`	Required integer $n_{\min}$ obtained before numerical NDE work from the symbolic boundary condition and DE. The literal `"automatic"` is rejected.
`leading_power_certificate`	Required mapping with exactly `status="certified"`, the same integer `leading_power`, and a nonempty `method`. FlintNDE validates this contract before any sample solve.
`series_parameter="ep"`	Name of the parameter reconstructed by the outer sampling loop.
`goal_digits=30`	Positive requested decimal accuracy g used by the automatic sampling, working-precision, and validation rules.
`sample_points="automatic"`	Automatic uses Eq. (53) and the rationalized one-percent grid. An explicit sequence bypasses automatic production point generation but not validation.
`sample_count="automatic"`	Number N_{fit} of production interpolation points. It must cover at least the powers $n_{\min}, \ldots, m$.
`base_sample="automatic"`	Production scale ϵ_0; automatic means $10^{-\alpha_\epsilon}$ from Eq. (53).
`sample_spacing=0.01`	Relative spacing δ in $\epsilon_j = \epsilon_0(1 + j\delta)$.
`working_precision_digits="automatic"`	Decimal precision of each NDE solve; automatic means p_{work} in Eq. (53).
`extra_working_precision=0.0`	Fractional safety increase in $p_{\text{work}} = \lceil 2(1 + \texttt{extra\_working\_precision})p_0 \rceil$; ignored when `working_precision_digits` is supplied.
`transport_order="automatic"`	Primary local-series order used by every underlying NDE transport; automatic means $4p_0$.
`transport_extra_order="automatic"`	Additional local-series order for the refinement solve; automatic means $\lceil \max(50, p_0/5) \rceil$.
`transport_sample_count="automatic"`	Cauchy–DFT circle samples for the primary NDE chain; automatic uses the basic solver rule $\max(32, 2n)$.
`transport_extra_sample_count="automatic"`	Cauchy–DFT circle samples for the reference chain, with the same automatic rule at the higher order.
`radius_fraction=0.60`	Fraction of the nearest-singularity distance used as each ordinary-point Cauchy radius.
`guard_bits=32`	Binary guard bits added after converting the decimal working precision to the global FLINT context.
`validation_sample_count=2`	Number N_{check} of solves excluded from the interpolation and reserved for a posteriori residual checks.
`validation_scale=0.5`	Scale of the independent validation grid relative to the production grid.
`validation_tolerance="automatic"`	Relative residual tolerance; automatic targets 10^{-g} while respecting Acb enclosure radii and reports the effective cutoff.

Interface or parameter	Meaning and constraints
`maximum_samples=100`	Cap on N_{fit}, matching the AMFlow production-sample cap. Validation evaluations are reported separately.
`rationalize_sample_points=True`	Convert automatically generated decimal scales and spacings to exact rational regulator values before each NDE solve. False passes Acb samples to the factories instead.
`parallel_task_count=12`	Maximum number of independent fixed-regulator worker processes. The effective count is `min(parallel_task_count, number of samples)`; queued samples start automatically as workers finish. Fixed exact rational systems, ordinary boundary vectors, and serialized adaptive paths are restored in each worker. Arbitrary `AnalyticMatrixSystem` instances and local closures must instead be module-level factories or run with count one.
`output_layout=None`	Optional caller-local `OutputLayout`. When supplied, the JSON result is written only under its `regularization` category.
`result_name="series_reconstruction"`	Single path-safe stem for the JSON filename.
`SeriesReconstructionResult`	Provides `powers`, coefficient column vectors, production and validation points/values, effective parameters, diagnostics, `coefficient(power)`, and `evaluate(ep)`.
Failure classes	`ValueError` rejects a missing or inconsistent leading-power certificate. `SeriesValidationError` rejects an unresolved NDE refinement, interpolation, or independent-sample residual.

6.7 Connection matrices and boundary subspaces

Let Φ_a and Φ_b be local fundamental matrices near two endpoints and let U_γ be ordinary-point transport along a path γ. Their connection matrix is

$$C_{ba}^{(\gamma)} = \Phi_b^{-1} U_\gamma \Phi_a. \tag{58}$$

If V_a spans the allowed initial boundary subspace and L_b annihilates the allowed final subspace, a nonzero two-end solution satisfies

$$L_b C_{ba}^{(\gamma)} V_a a = 0. \tag{59}$$

This form covers both master-integral boundary matching and spectral boundary-value problems.

7 Software status and outlook

The development release depends only on python-flint>=0.6 and contains the general Cauchy–DFT ordinary-point backend, exact or precision-threshold-gated power–log bases, exact $\{a, b, C\}$ singular boundary initialization, exact Lee–Moser projector balances, certified decoupled exponential sectors, start-only simple-spectrum formal asymptotics with fixed-order five-term diagnostics, regulator reconstruction with user overrides, unit tests, and the two-dimensional boundary example. A 16-dimensional complex-exact test has also exercised the general `RationalMatrixSystem` over $\mathbb{Q}(i)(k)$: it discovered seven finite singularities and recovered the exact roots $0, 9$, fourteen active resonance gates, and maximum log degree one. The next release milestones are a public flat-space Feynman-integral example, same-input benchmarks against specialized rational coefficient generators, an installable command-line interface, repeated-root formal block decoupling, ramification, algebraic extensions, and Stokes matching. The package will remain fail closed whenever local branch data are insufficient.

References

[1] K. G. Chetyrkin and F. V. Tkachov. Integration by parts: The algorithm to calculate β-functions in 4 loops. *Nucl. Phys. B*, 192:159–204, 1981.

[2] T. Gehrmann and E. Remiddi. Differential equations for two-loop four-point functions. *Nucl. Phys. B*, 580:485–518, 2000.

[3] Martijn Hidding. DiffExp, a Mathematica package for computing Feynman integrals in terms of one-dimensional series expansions. *Comput. Phys. Commun.*, 269:108125, 2021.

[4] Xiao Liu and Yan-Qing Ma. AMFlow: A Mathematica package for Feynman integrals computation via auxiliary mass flow. *Comput. Phys. Commun.*, 283:108565, 2023.

[5] Rui-Jun Huang, Xiao Liu, and Yan-Qing Ma. AMFlow 2.0: significant algorithmic and software improvements for Feynman integral evaluation. 7 2026.

[6] Wen Chen. Semi-automatic calculations of multi-loop Feynman amplitudes with AmpRed. *Comput. Phys. Commun.*, 312:109607, 2025.

[7] Roman N. Lee and Andrei A. Pomeransky. Critical points and number of master integrals. *JHEP*, 11:165, 2013.

[8] M. Beneke and Vladimir A. Smirnov. Asymptotic expansion of Feynman integrals near threshold. *Nucl. Phys. B*, 522:321–344, 1998.

[9] A. Pak and A. Smirnov. Geometric approach to asymptotic expansion of Feynman integrals. *Eur. Phys. J. C*, 71:1626, 2011.

[10] Xingang Chen and Yi Wang. Large non-Gaussianities with Intermediate Shapes from Quasi-Single Field Inflation. *Phys. Rev. D*, 81:063511, 2010.

[11] Xingang Chen and Yi Wang. Quasi-Single Field Inflation and Non-Gaussianities. *JCAP*, 04:027, 2010.

[12] Nima Arkani-Hamed and Juan Maldacena. Cosmological Collider Physics. 3 2015.

[13] Jiaqi Chen and Bo Feng. Towards systematic evaluation of de Sitter correlators via Generalized Integration-By-Parts relations. *JHEP*, 06:199, 2024.

[14] Jiaqi Chen, Bo Feng, and Yi-Xiao Tao. Multivariate hypergeometric solutions of cosmological (dS) correlators by d log-form differential equations. *JHEP*, 03:075, 2025.

[15] Jiaqi Chen, Bo Feng, Zhehan Qin, and Yi-Xiao Tao. Loop integrals in de Sitter spacetime: The parity-split IBP system and d log-form differential equations. 4 2026.

[16] Nima Arkani-Hamed, Daniel Baumann, Aaron Hillman, Austin Joyce, Hayden Lee, and Guilherme L. Pimentel. Differential equations for cosmological correlators. *JHEP*, 09:009, 2025.

[17] Nima Arkani-Hamed, Daniel Baumann, Aaron Hillman, Austin Joyce, Hayden Lee, and Guilherme L. Pimentel. Kinematic Flow and the Emergence of Time. *Phys. Rev. Lett.*, 135(3):031602, 2025.

[18] Shounak De and Andrzej Pokraka. Cosmology meets cohomology. *JHEP*, 03:156, 2024.

[19] Daniel Baumann, Austin Joyce, Hayden Lee, and Kamran Salehi Vaziri. Differential Equations for Massive Correlators. 4 2026.

[20] Kostas Glampedakis, Aaron D. Johnson, and Daniel Kennefick. Darboux transformation in black hole perturbation theory. *Phys. Rev. D*, 96(2):024036, 2017.

[21] E. W. Leaver. An Analytic representation for the quasi normal modes of Kerr black holes. *Proc. Roy. Soc. Lond. A*, 402:285–298, 1985.

[22] Emanuele Berti, Vitor Cardoso, and Andrei O. Starinets. Quasinormal modes of black holes and black branes. *Class. Quant. Grav.*, 26:163001, 2009.

[23] Francesco Moriello. Generalised power series expansions for the elliptic planar families of Higgs + jet production at two loops. *JHEP*, 01:150, 2020.